PROGRAMMING IN JAVA

CSA0915

ASSIGNMENT-CO1

CAMPUS SMART-PARKING SYSTEM

TASK 3 – Design the Control Logic

Name: V. Reddy Sreya **Regno: 192525124**

1. Task03.md

1. Slot Allocation Logic

The system searches each floor and its slots using nested for loops.

- If a slot is empty (null), the vehicle is allocated.
- The allocated floor and slot numbers are returned.
- The search stops using return.
- If all slots are occupied, the method returns null.

2. Parking Fee Calculation

The parking fee is calculated based on the parking duration.

Example fee rules (assumed):

- First hour: Rs. 30
- Each additional hour: Rs. 20
- Daily maximum: Rs. 200

The billable hours are calculated using:

hours = (minutes + 59) / 60

An if-else statement can be used to apply the fee slabs.

3. Use of break, continue and return

- **break:** Stops the loop when a suitable slot is found.
- **continue:** Skips an occupied slot and checks the next slot.
- **return:** Returns the allocated slot or stops the method when no slot is available.

4. When No Slot Is Available

If all parking slots are occupied, the allocate() method returns null. The system displays a message that no parking slot is available.

5. Pseudocode – Slot Allocation

```
BEGIN
  FOR each floor
    FOR each slot
      IF slot is empty
        Allocate vehicle
        RETURN floor and slot
      END IF
    END FOR
  END FOR
  RETURN NULL
END
```

6. Pseudocode – Fee Calculation

```
BEGIN
  READ parking duration
  IF minutes <= 0
    fee = 0
  ELSE
    hours = (minutes + 59) / 60
    IF hours == 1
      fee = 30
    ELSE
      fee = 30 + (hours - 1) * 20
    END IF
    IF fee > 200
      fee = 200
    END IF
  END IF
  DISPLAY fee
END
```

7. Alternative Concept and Complexity Analysis

Instead of searching every slot, a stack of free slot positions can be maintained for each floor.

Operation	**Nested Loop**	**Stack of Free Slots**
Allocation	O(F × S)	O(1)
Release	O(1)	O(1)
Extra space	O(F × S) for slots	O(F × S) for free-slot stacks

Here, F represents the number of floors and S represents the maximum slots per floor.

Conclusion:

The nested-loop approach is simple and suitable for a small parking facility. A stack of free slots provides faster allocation for a large parking facility.

2.ParkingFacility.java

```
public class ParkingFacility {

    private final String[][] slots;

    public ParkingFacility(int[] slotsPerFloor) {

        slots = new String[slotsPerFloor.length][];

        for (int f = 0; f < slotsPerFloor.length; f++) {

            slots[f] = new String[slotsPerFloor[f]];

        }

    }

    // Allocate the first available slot

    public int[] allocate(String vehicleNo) {

        for (int f = 0; f < slots.length; f++) {

            for (int s = 0; s < slots[f].length; s++) {


                if (slots[f][s] == null) {

                    slots[f][s] = vehicleNo;

                    return new int[]{f, s};

                }

            }

        }
```

```
    return null;
}
// Release an occupied slot
public boolean release(int floor, int slot) {
    if (floor < 0 || floor >= slots.length) {
        return false;
    }
    if (slot < 0 || slot >= slots[floor].length) {
        return false;
    }
    if (slots[floor][slot] == null) {
        return false;
    }
    slots[floor][slot] = null;
    return true;
}
public static void main(String[] args) {
    ParkingFacility p =
            new ParkingFacility(new int[]{2, 1});
    System.out.println(java.util.Arrays.toString(
            p.allocate("TN01AB1111")));
    System.out.println(java.util.Arrays.toString(
            p.allocate("TN02CD2222")));
```

```
        System.out.println(java.util.Arrays.toString(
                p.allocate("TN03EF3333")));
        System.out.println(java.util.Arrays.toString(
                p.allocate("TN04GH4444")));
        System.out.println(
                p.release(0, 1) + " " +
                p.release(0, 1) + " " +
                p.release(5, 0));
        System.out.println(java.util.Arrays.toString(
                p.allocate("TN04GH4444")));
    }
}
```

3. Expected output

